

3W Toolkit: A Set of Tools for Oil & Gas Data Processing and Analysis

Ricardo E. V. Vargas¹, Afrânio J. M. Junior², Rafael Padilla¹, Thadeu L. B. Dias¹, Matheus E. Santo¹, Eduardo H. Banaczewski¹, Pedro H. B. Lisboa¹, Gabriel H. B. Lisboa¹, Luiza H. de A. Leite¹, Matheus R. Parracho¹, and Bruno C. Martins¹

¹ Universidade Federal do Rio de Janeiro (UFRJ), Brazil ² Petrobras, Brazil

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 17 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

The **3W Toolkit** is an open-source tool for time series processing, providing early undesirable event detection and diagnosis in oil well operations.

It has a modular architecture covering data preprocessing, feature extraction, dimensionality reduction, model training, performance evaluation, and graphical outputs. Designed for oil and gas professionals who need efficient tools to explore large production and exploration datasets. The **3W Toolkit v3.0.0** is available as a Python package with documentation and example workflows for integration with existing systems.

It targets early automatic detection and classification of failure events in oil and gas wells and pipelines, as depicted in [Figure 1](#). The considered events belong to the public **3W Dataset** ([3W project Petrobras, 2019](#); [Vaz Vargas et al., 2026](#)), developed by Petrobras ([Petrobras, 2026](#)), a Brazilian oil holding company. The **3W Dataset** is the project reference dataset and is hosted on Figshare([Figshare, 2026](#)).

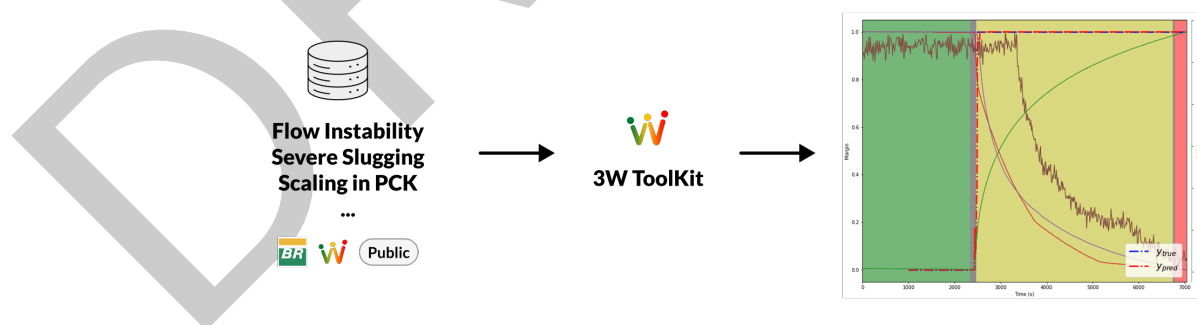


Figure 1: 3W Toolkit: Open-source tools for time series processing.

Statement of need

Prompt corrective actions help avoiding costly production-well interventions, making timely fault identification essential. Petrobras's public **3W Dataset** ([3W project Petrobras, 2019](#)) documents fault types from oil-well operations. This pioneering dataset helped transform the oil and gas industry by providing the first public, realistic dataset containing real undesirable oil-well events.

27 The **3W Toolkit** is part of the **3W** project developed by Petrobras (Petrobras, 2026), the Signal,
28 Multimedia and Telecommunications Laboratory (SMT)(SMT, 2026), and Signal Processing
29 Laboratory (LPS)(LPS, 2026) at the Federal University of Rio de Janeiro (UFRJ). It provides
30 tools to process and analyze large volumes of oil and gas exploration and production data,
31 including well analysis and fault detection.

32 The **3W Toolkit** addresses the need for integrated, accessible tools for professionals handling
33 large data volumes. Its modular framework supports multiple data preprocessing techniques,
34 feature extraction methods, classifiers, and performance metrics. It provides a common
35 comparison ground, avoiding incompatible experiments across researchers and companies. It
36 also gives beginners in the **3W Community** (3W community, 2019) a ready-made package for
37 exploring the **3W Dataset**.

38 Developed in Python, the toolkit integrates easily with Python-based systems and data analysis
39 workflows. As open-source software, it also supports community collaboration and improvement.

40 The **3W Toolkit** has evolved across architecture, software design, data structures, and
41 features. Among other improvements, the current version currently processes .parquet
42 files for better memory efficiency and faster queries, streamlines large-scale workflows, and
43 includes machine learning features for time-series anomaly detection and event classification.
44 It now uses a modular, object-oriented package with dedicated sub-modules. Unlike the initial
45 standalone scripts and Jupyter Notebooks for parsing raw sensor streams, the current release
46 is a standardized package managed through modern dependency structures (pyproject.toml)
47 and installed via PyPI.

48 State of the field

49 Petrobras launched the **3W Community** (3W community, 2019), an international collaboration
50 of researchers, startups, companies, and independent data scientists developing artificial
51 intelligence and machine learning tools for early offshore oil-well event detection. With
52 the widespread adoption of the **3W Dataset** (3W project Petrobras, 2019) among research
53 institutions, the **3W Toolkit** was designed to streamline these efforts. While time-series toolkits
54 (e.g., sktime, tsfresh) provide foundational algorithms, they lack domain-specific knowledge
55 of oil-well variables, multi-source telemetry differences (simulated vs. real events), and 3W
56 operational constraints. Rather than competing with general-purpose tools, the **3W Toolkit**
57 connects raw sensor data to standard machine learning paradigms, keeping comparisons within
58 the 3W Community reproducible and aligned with industry standards.

59 There are currently no other specialized Python packages, frameworks, or toolkits built
60 specifically to address the heterogeneous composition and multi-source nature of the **3W**
61 **Dataset**. Without a unified framework like the **3W Toolkit**, researchers must create isolated
62 ingestion scripts, causing inconsistent handling of real, simulated, and hand-drawn synthetic
63 instances. The **3W Toolkit** therefore consolidates one-off scripts into a reproducible platform.

64 Software design

65 Modularity is a cornerstone of the project. Each component operates independently and can
66 be used, replaced, or updated without affecting others. This architecture improves flexibility
67 and scalability, enabling customization and expansion. Reusable modules simplify maintenance
68 because issues can be fixed locally. Modules can also be combined into tailored solutions, while
69 new ones can be added without major restructuring.

70 The **3W Toolkit** modularity supports updates, while documentation improves accessibility. Its
71 broad functionality encourages adoption by more users. The schema in Figure 2 shows the
72 toolkit's main classes.

The architecture has two abstraction layers: *Core* and *Application*. The *Core* layer defines fundamental abstractions and includes *BaseDataset*, *BasePreprocessing*, *BaseFeatureExtractor*, *BaseModels*, *BaseTrainer*, and *BasePipeline*. These provide consistent interfaces that keep implementations interchangeable. Lightweight containers such as *DatasetOutputs*, *TrainingResult*, *PredictionResult*, and *AssessmentOutput* standardize communication between modules, reduce coupling, and improve result traceability.

The *Application* layer implements these abstractions. *ParquetDataset* handles structured dataset loading, while *Normalize* and *Windowing* provide preprocessing and feature extraction. Models are divided into deep learning, represented by *TorchModels*, and traditional machine learning, encapsulated by *SklearnModels*. This separation supports heterogeneous modeling through one interface.

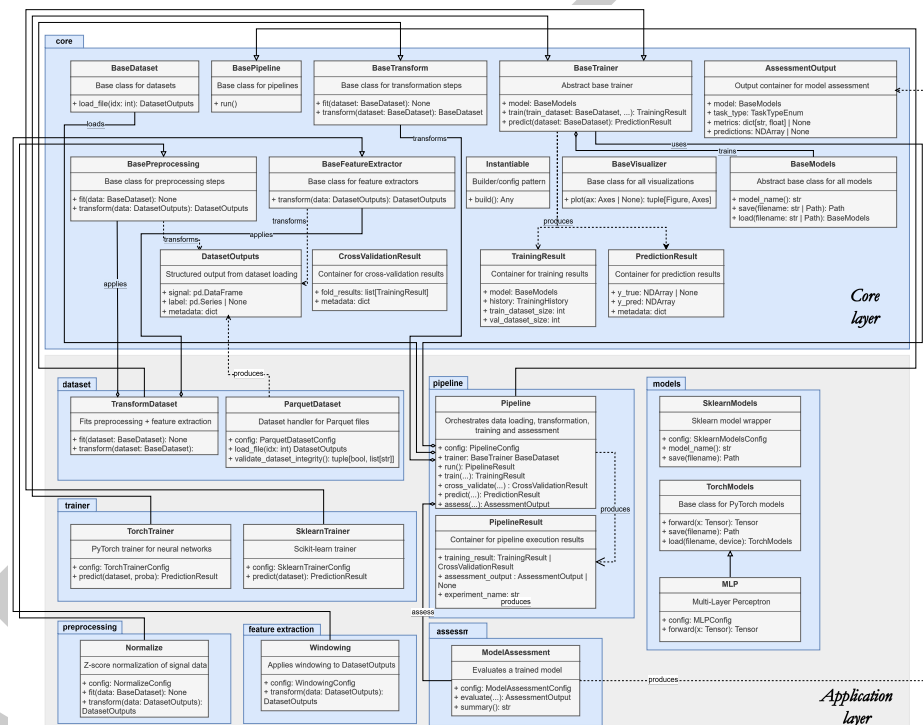


Figure 2: Toolkit schema.

Training uses specialized classes such as *TorchTrainer* and *SklearnTrainer*, both derived from *BaseTrainer*. Separating training logic from models enables reuse across models. Evaluation uses *ModelAssessment*, which produces standardized outputs independent of model type.

Pipeline orchestrates dataset loading, preprocessing, feature extraction, training, prediction, and assessment. Encapsulating these steps in one configurable component enables reproducible experiments and simplifies workflows such as cross-validation and performance evaluation.

Finally, configuration-driven components and instantiation patterns make experiments easy to reproduce and modify. This design supports both research and production environments.

Installation

The **3W Toolkit** is currently distributed as part of the 3W Project repository. The source code can be obtained from: <https://github.com/petrobras/3W.git>.

95 The toolkit is located in the `toolkit/ThreeWToolkit` directory. It is recommended to install
96 it within an isolated Python environment.

97 For example, using `uv`:

```
uv venv                                # Create virtual environment
source .venv/bin/activate               # On Linux/macOS
.venv\Scripts\activate                  # On Windows

cd toolkit/ThreeWToolkit
uv pip install -e .
```

98 Alternatively, installation can be performed using `pip`:

```
pip install -e .
```

99 This will install the toolkit in editable mode, allowing users to modify and extend its components.

100 Features

101 The **3W Toolkit** provides a modular, extensible framework for time-series fault detection and
102 classification in oil well operations. Its main capabilities include:

- 103 ▪ **Dataset handling and filtering.** Utilities load structured datasets (e.g., Parquet) and
104 filter by event type, target classes, and custom file lists, enabling reproducible splits and
105 controlled experiments.
- 106 ▪ **Preprocessing pipelines.** Reusable components provide signal cleaning, missing value
107 imputation, normalization, label handling, and column transformations. They can be
108 composed into sequential pipelines for consistent training and inference.
- 109 ▪ **Feature extraction for time-series data.** Window-based strategies include statistical
110 descriptors, exponentially weighted statistics, and wavelet-based features, which can be
111 combined into richer temporal representations.
- 112 ▪ **Visualization and exploratory analysis.** Built-in utilities inspect individual signals, compare
113 series, and analyze correlations, helping users understand data before modeling.
- 114 ▪ **Model training with heterogeneous backends.** The framework supports deep learning
115 (via PyTorch) and traditional machine learning (via Scikit-learn), providing a unified
116 interface for training, prediction, and model persistence.
- 117 ▪ **Pipeline-based workflow orchestration.** An integrated abstraction defines end-to-end
118 workflows covering loading, preprocessing, feature extraction, training, and evaluation,
119 improving reproducibility and reducing boilerplate.
- 120 ▪ **Evaluation and reporting.** The toolkit provides standardized evaluation outputs, multiple
121 metrics, and automated HTML and LaTeX reports for sharing and documentation.
- 122 ▪ **Experiment reproducibility.** Configuration-driven components and explicit dataset
123 splitting, preprocessing, and model parameters enable consistent reproduction and
124 comparison of experiments.

125 Example Usage

126 The following example shows a minimal **3W Toolkit** workflow for dataset loading, preprocessing,
127 feature extraction, model training, and evaluation.

```
from ThreeWToolkit.dataset import ParquetDatasetConfig, TransformConfig
from ThreeWToolkit.preprocessing import (
```

```

CleanSignalsConfig,
ImputeMissingConfig,
NormalizeConfig,
SequentialPreprocessingAdapterConfig,
)
from ThreeWToolkit.feature_extraction import (
    WindowingConfig,
    StatisticalConfig,
    SequentialFeatureAdapterConfig,
)
from ThreeWToolkit.models import MLPConfig
from ThreeWToolkit.trainer import TorchTrainerConfig
from ThreeWToolkit.assessment import ModelAssessmentConfig

# Load dataset
dataset = ParquetDatasetConfig(path="./dataset").build()

# Define preprocessing + feature extraction pipeline
transform = TransformConfig(
    pre_processing=SequentialPreprocessingAdapterConfig(
        steps=[
            CleanSignalsConfig(),
            ImputeMissingConfig(),
            NormalizeConfig(),
        ]
    ),
    feature_extraction=SequentialFeatureAdapterConfig(
        steps=[
            WindowingConfig(window_size=128),
            StatisticalConfig(),
        ]
    ),
).build()

# Transform
transform.fit(dataset)
dataset_transformed = transform.transform(dataset)

# Define model and trainer
trainer = TorchTrainerConfig(
    config_model=MLPConfig(hidden_sizes=(32, 16)),
    epochs=10,
).build()

# Train model
training_result = trainer.train(dataset_transformed)

# Predict and evaluate
predictions = trainer.predict(dataset_transformed)

assessment = ModelAssessmentConfig(metrics=["accuracy"]).build()
results = assessment.evaluate(
    training_results=training_result,
    predictions=predictions,

```

)

```
print(results.metrics)
```

128 Data Visualization

129 The 3W Toolkit provides a data visualization module (DataVisualization) for graphical
130 temporal-series analysis, helping users explore, interpret, and communicate data patterns.

131 [Figure 3](#) shows a correlation heatmap generated by the **3W Toolkit**, while [Figure 4](#) shows
132 temporal signal plots. These visualizations support sensor comparison and analysis of
133 relationships among measured variables.

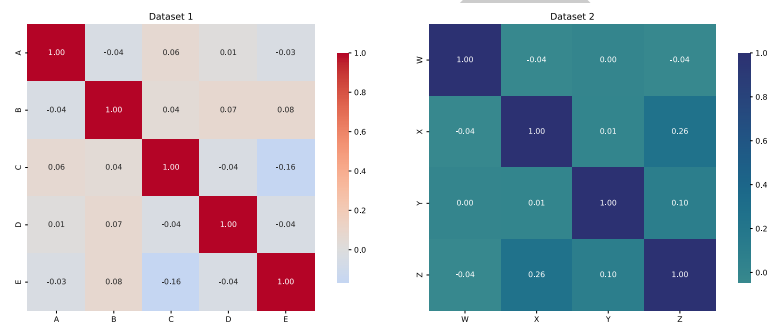


Figure 3: Correlation heatmap of sensor measurements.

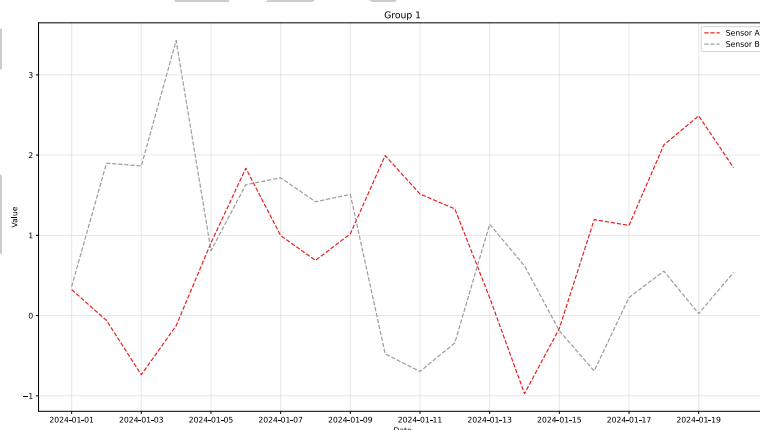


Figure 4: Temporal signals collected from multiple sensors.

134 Research impact statement

135 The **3W Toolkit** bridges industry and academia through collaboration between Petrobras and
136 the Federal University of Rio de Janeiro (UFRJ). Researchers recently used **3W Toolkit** (v3.0.0)
137 to standardize the 3W Dataset pipeline with automated cleaning, temporal alignment, and
138 class selection for robust loading ([Andrade et al., 2026](#); [Pessoa et al., 2026](#)). Standardized
139 selection, filtering, and loading routines ensure reproducibility and compliance with dataset-
140 maintainer quality criteria. The toolkit provides an industry-validated sandbox, allowing
141 academic contributions to be tested against real-world domain constraints, directly advancing
142 the state of the art in well integrity and flow assurance automation.

Conclusions

The **3W Toolkit** is an open-source, modular framework for fault detection and classification in oil well operations. It flexibly integrates preprocessing, feature extraction, modeling, and evaluation for research and practical applications. Its unified pipeline and multiple modeling approaches support reproducible end-to-end machine learning workflows for time-series data. Jupyter notebooks provide step-by-step guidance for its features. Although developed around the **3W Dataset**, the toolkit can be adapted to other datasets.

AI usage disclosure

This project used GitHub Copilot and Claude for documentation purposes, and all contributions were carefully reviewed by multiple authors for consistency and accuracy.

References

- 3W community*. (2019). <https://github.com/petrobras/3W/blob/main/community/README.md>.
- 3W project Petrobras. (2019). *3W project*. <https://github.com/petrobras/3W>.
- Andrade, L. H. de, Moura Jr., N. N. de, Netto, S. L., Vargas, R. E. V., & Melo Jr., A. J. de. (2026). Operadores Neurais e Aprendizado Profundo para Classificação de Falhas em Poços de Petróleo (Neural Operators and Deep Learning for Fault Classification in Oil Wells). *Anais Do XXVI Congresso Brasileiro de Automática (CBA 2026)*.
- Figshare. (2026). <https://figshare.com/>.
- LPS. (2026). <https://www.pee.ufrj.br/lps-laboratorio-de-processamento-de-sinais/>.
- Pessoa, J. V. C., Chaves, R. S., & Campos, M. L. R. de. (2026). Multivariate time series classification in resource-constrained embedded systems. *Proceedings of the XLIV Brazilian Symposium on Telecommunications and Signal Processing (SBrT 2026)*.
- Petrobras. (2026). <https://petrobras.com.br/>.
- SMT. (2026). <https://www.smt.ufrj.br/en/>.
- Vaz Vargas, R. E., Melo Junior, A. J. de, Campos Lima, C. B. de, Araújo, J. C. D. de, Araujo Fernandes, M. de, Alves Pinto, R. L., Barbosa, J. R., Munaro, C. J., Ciarelli, P. M., Varejão, F. M., Barrocas, F. M., Carneiro, J. N. E., Scramignon, M. L., Branco, R. C., Siqueira, T. P., Lima Junior, E. T. de, Melo Nery Oliveira, I. de, Lopes, L. G. O., Gouveia, L. P. de, ... Cadena, J. A. L. (2026). 3W dataset 2.0.0: A realistic and public dataset with rare undesirable real events in oil wells. *Scientific Data*. <https://doi.org/10.1038/s41597-026-07225-z>